Steer, Don't Solve: Training Small Critic Models for Large Coding Agents

Shubham Gandhi*[*], Yiqing Xie*, Atharva Naik, Ruichen Zhu, Carolyn Rosé
Carnegie Mellon University
{srgandhi, yiqingxi, arnaik, rzhu3, cp3a}@andrew.cmu.edu

Abstract

Coding tasks are typically complicated and require multiple capabilities, ranging from high-level planning to low-level implementation. While coding agents are optimized for the joint capabilities, individual capabilities such as high-level planning may have different optima and remain a major bottleneck. To address this challenge, we train a separate critic model that is *specialized in high-level planning* to steer the coding agent in inference. We construct SFT and DPO data to train the critic model to identify errors made by the coding agent and provide correct and clear high-level guidance without generating concrete actions. Experiments show that our fine-tuned 4B and 8B critic models significantly improve the performance of 6 larger coding agents (e.g., improving the resolved rates of `GLM-4.7-Flash-30B-A3B` and `GPT-OSS-120B` by 16.0% and 14.4% on SWE-Bench Verified). The critic model also reduces the total inference costs for some coding agents by solving tasks in fewer steps (e.g., reducing the per-example inference cost for `GPT-OSS-20B` from $0.07 to $0.03). [1]

1 Introduction

Coding agents have been widely used in real-world programming tasks (Jimenez et al., 2024; Yang et al., 2024), and post-training has proven effective for coding agent training (Pan et al., 2025; Yang et al., 2025b; Xie et al., 2026), where the model is optimized for the combination of multiple abilities, including high-level reasoning (e.g., troubleshooting, search, planning) and low-level implementation (executing tool calls correctly).

Theoretically, the optima for the joint capabilities could be different from the optima for each individual capability, such as high-level planning.

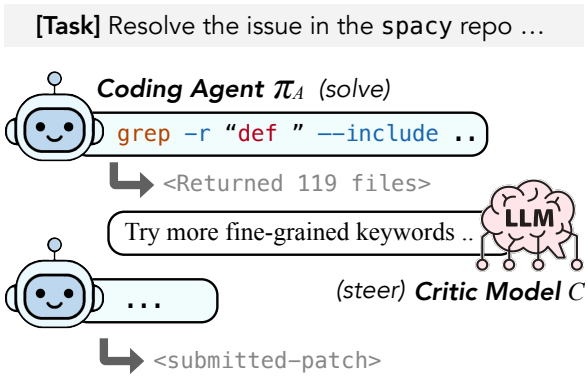


Figure 1: Inference setup. A trained critic model C provides guidance to a frozen coding agent π_A for every k steps. The agent is responsible for all concrete actions and the critic model inspects the current trajectory and generates high-level feedback for its next steps.

Empirically, during post-training, multiple studies have shown that the planning ability of a coding agent is still far from perfect after reaching the plateau on the training data (Xie et al., 2026; Wang et al., 2026a). For example, about 65% of coding agent failures come from flawed reasoning, not wrong actions (Liu et al., 2025). A possible explanation is that integrating both high-level and low-level knowledge in training makes the learned patterns overly complex, which undercuts the coding agent's ability to learn generalizable patterns and limits the overall performance.

To address this challenge, we shift part of the high-level planning role to a separate, specialized **critic model**, which steers the main coding agent by providing feedback during inference. The idea is inspired by research on critic and adviser models in other NLP domains such as question-answering (Gou et al., 2023), factuality (Xie et al., 2025), and mathematical problems (Lightman et al., 2024). Compared to training the coding agent itself, we highlight the advantages of training a critic model as follows: First, the critic model im-

[*]Equal contribution.
[1]Code: https://github.com/shubhamrgandhi/critic-training.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 56259–56276
October 24-29, 2026 ©2026 Association for Computational Linguistics

proves the overall performance by complementing the planning ability of the coding agent. Second, critic model training is cost-efficient in terms of both data curation and computational resources. §2 shows that we can re-use existing coding-agent environments for critic model training, without building new training tasks or environments. §3.3 shows that to improve the overall performance, we can train a small critic model instead of training the large coding agent, which requires fewer computational resources. Finally, the use of critic models is highly adaptive and flexible. We can provide critiques to any frozen coding agent at any step with any frequency.

In this paper, we design a two-stage training framework to train the critic models. The first training stage (Figure 2) uses supervised fine-tuning (SFT), where we distill from a strong teacher model (e.g., `Claude-Opus-4.6`). We explicitly require the target critiques to detect the major errors made by the coding agent and provide corresponding high-level guidance to the next steps, without containing any concrete implementation such as specific tool-call commands. As shown in our analyses, there is still a substantial gap between the overall guidance generated by the SFT student model and the teacher critic model. Therefore, we present the second training stage with direct preference optimization (DPO), where we construct preference pairs based on the quality of the guidance in the critiques. Specifically, we design two metrics: correctness and clarity. Correctness requires the critiques to ground to the previous error categorization results and suggest the correct direction for future steps. Clarity checks whether the critiques provide clear instructions without ambiguity or under-specification.

Experiments on SWE-Bench Verified (Jimenez et al., 2024) show that our trained critic model improves the resolved rate of six different coding agent models by 5.6%–16.0%, without re-training the coding agents. We further demonstrate that our training can transfer across different programming languages and coding agents in training and evaluation. For instance, we improve the performance on Multi-SWE-Bench (Zan et al., 2025) by 2.6%–3.6% on 7 unseen languages. We also conduct detailed analyses to demonstrate the effectiveness of the SFT stage, DPO stage, and the design choice of training on high-level critiques only. Finally, we provide qualitative analyses and case studies on the critiques generated by our models.

2 Methodology

Long-horizon coding agents drift: they overcommit to early hypotheses, repeat failed commands, or allocate steps to subgoals in a suboptimal manner (Gandhi et al., 2025; Wang et al., 2026a). To overcome this challenge, we leverage a critic as a second model to inspect the trajectory at runtime in order to alert the model to such drift. Empirically, pairing a frozen coding agent with a strong proprietary model such as `Claude-Opus-4.6` (Anthropic, 2026) as a critic substantially improves the performance but also requires a higher total cost. For instance, it improves the resolved rate of `Qwen3-32B` on SWE-Bench-Verified from 8.8% to 26.8%, but also increases the per-instance cost from $0.05 to $0.28.

To achieve a better balance between cost and performance, in this section, we train a smaller open-source critic model to guide the coding agent during inference. Specifically, we first introduce the formal definitions (§2.1) and the design of the critique (§2.2). Then we design training data for the SFT (§2.3) and DPO stages (§2.4).

2.1 Formal Definitions

Coding Agent. Given a coding task and a docker environment in which to perform the task, the coding agent has access to a set of tools (e.g., the bash command-line tool) that can be called through tool-call actions. In problem-solving, the agent generates a trajectory as the solution $\tau_{1:t} = (a_1, o_1, \ldots, a_t, o_t)$, where a_i is the agent's action and o_i the corresponding environment observation. We limit the number of steps with a fixed budget $t \leq B$. In our setting, π_A is held frozen throughout and we only train the critic model C_θ.

Critic Models for Coding Agents. Formally, we pair the coding agent π_A with a critic model C at inference time. For every k agent steps (we use $k = \{5, 10\}$ in our experiments), the critic model inspects the task and the current trajectory prefix $\tau_{1:t}$. To preserve continuity across interventions, we also include the feedback the critic produced at earlier intervention points. Let $h_t = (f_k, f_{2k}, \ldots, f_{t-k})$ denote the critic's prior feedback history before step t. We form the critic input $z_t = \text{Serialize}(\tau_{1:t}, h_t, x)$, which fits the trajectory to the context window limit x by preserving the start of the trajectory (system prompt and task context) and the most recent steps, including the final user message, and finally truncating the in-

termediate steps that exceed the context limit. We sample feedback $f_t \sim C(\cdot \mid z_t)$, which is appended to the agent's context for the next step.

2.2 Designing the Critique Structure

We design the structure of the critique as follows: it first outputs the major errors made by the coding agent and the recovery action, where the error categories are pre-defined. Then it outputs the overall guidance, which contains only high-level feedback instead of the concrete tool call actions. Finally, it outputs a reminder when the agent is close to using up the step limit. We provide the prompt for critique generation in App. E.1.

Error Detection for the Coding Agent. Multiple existing studies show that coding agents' capability to recover from errors is still limited (Wang et al., 2026a; Gandhi et al., 2025). One example is the "stuck-in-loop" behavior, where the agent keeps generating the same tool call action even if it fails every time. To improve error recovery, we prompt the critic model to detect major errors made by the coding agent. Inspired by SWE-PRM (Gandhi et al., 2025), we pre-define 12 trajectory-level error categories covering *Specification*, *Reasoning*, and *Coordination* errors. The complete list is provided in App. E. For each category, the critic model reports whether the error is detected, the supporting evidence, recovery strategies, and a task-status verdict showing whether the error is not significant and the agent is still on track, or the error is serious and needs correction or critical intervention.

High-level Feedback Only, NOT Concrete Actions. The motivation for critic model training is to separate the high-level planning ability to a **specialized** model. As a result, we explicitly prompt the critic model NOT to suggest specific technical implementation or code-level changes, such as outputting the exact tool call command. To improve correctness, we also require the suggested recovery strategies or overall guidance to be grounded in previous error detection results. We additionally prompt the critic model to keep the guidance high-level (1-2 sentences per detected error), without giving too complicated instructions to coding agents that cannot be completed in a few steps. The prompt thereby restricts the teacher to high-level signals such as whether the agent is making progress, looping, relying on an unsupported assumption, or pursuing the wrong subgoal (App. E).

Budget Awareness. A high-level critic that ignores the budget will keep recommending exploration even when the agent is close to running out of steps, leaving useful in-progress work unsubmitted. Therefore, in inference, once the agent passes step T (e.g., we use T=100 of the B=150-step budget), we append short budget-aware instructions that state the steps remaining and recommend the coding agent to make a submission as soon as possible (App. E.3).

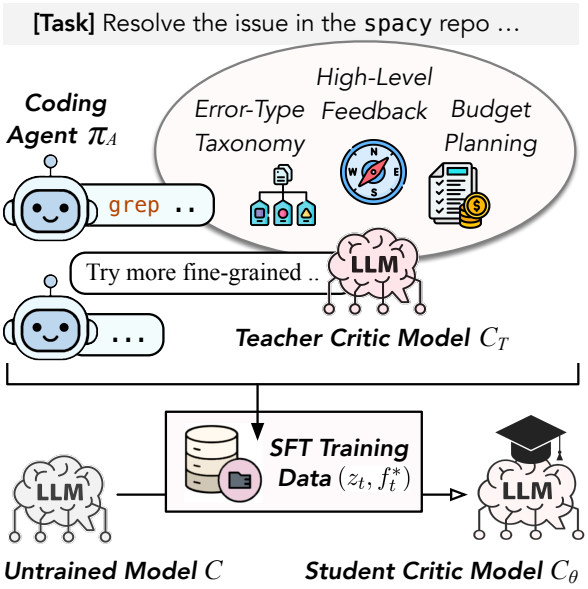


Figure 2: SFT training pipeline for critic models. Based on our analysis (§2.2), the critic contains error detection, budget planning, and only provides high-level feedback instead of concrete actions. We run the frozen coding agent π_A on disjoint training tasks and query the teacher critic C_T every k steps. We collect (trajectory prefix, feedback) pairs (z_t, f_t^*) to train a critic C_θ by supervised fine-tuning (SFT).

2.3 Distilling Critic Models with SFT

SFT Data Construction. As show in Figure 2, we prompt the teacher critic model C_T to provide guidance to a frozen coding agent π_A every k steps based on the current trajectory z_t on the training dataset $\mathcal{TR}_{\mathcal{SFT}}$ (as described in §3.1) and fine-tune a student critic model C_θ with the teacher's outputs as targets.

Based on the prompt design introduced in §2.2, we prompt the teacher model to first detect errors made by the coding agent and then provide overall guidance, with budget-aware alerts. We additionally require it to only generate high-level feedback rather than action-level prescriptions. We filter out malformed teacher responses (empty completions or responses that drop out of the critique format)

before training (§3.1). Note that we do NOT filter training critiques based on whether the task is successfully completed, which depends more on the coding agent itself rather than the critic model.

Critic Model Training with SFT. As shown in Figure 2, we distill C_θ from C_T by supervised fine-tuning (SFT) with a standard conditional language modeling objective, maximizing likelihood:

$$\max_\theta \mathbb{E}_{(z_t, f_t^*) \sim \mathcal{TR}_{\mathcal{SFT}}} \log \mathbb{P}_\theta(f_t^* \mid z_t). \quad (1)$$

The trained critic model C_θ is then paired with π_A at inference..

2.4 Improving Critic Models with DPO

The Remaining Gap after Distillation. We empirically study the gap between the student and teacher critic models after SFT (§2.3). As shown in App. A, the major gap exists in the **overall guidance** provided by the critic, instead of the error categorization. Among the 132 critiques we sampled, 67% of the teacher model's critiques both point to the correct direction and contain specific guidance for the next steps. In contrast, only 29% of the SFT student model's critiques are both correct and specific. 53% point to the correct direction but are under-specified. 18% suggest the wrong steps to take. We use an LLM judge for the judgment. We choose a different model from the teacher model to avoid self-bias (Zheng et al., 2023). We thus train the critic model for the second stage, focusing on generating high-quality overall guidance.

DPO Data Construction. We design two metrics to judge the quality of the guidance: (1) correctness, where the guidance fixes the serious error made by the coding agent (if any) and assists problem-solving; and (2) clarity, where the guidance is not ambiguous or under-specified. We adopt direct preference optimization (DPO) to improve these two metrics.

As shown in Figure 3, we apply the coding agent π_A on the training dataset TR_{DPO}. For every k steps, we use the SFT critic model $C_{SFT,\theta}$ to generate $N = 10$ critiques, with the previous trajectory z_t as context. Then we prompt an LLM judge to select the best and worst ones based on the correctness and clarity of the overall guidance. This forms one (chosen, rejected) training pair (f_c, f_r) for every set of critiques. We only keep the training pairs where the error categorization does not have major error, the chosen critique is both correct and clear, and the rejected critique is either not correct or not clear.

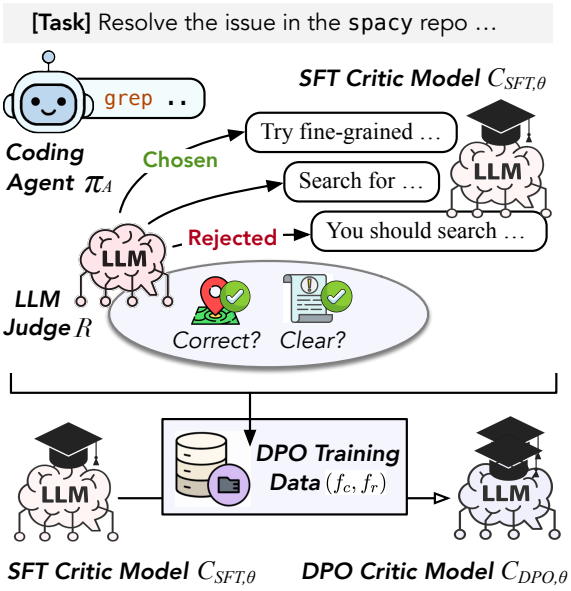


Figure 3: DPO training pipeline for critic models. Given a critic model checkpoint $C_{SFT,\theta}$, for every k steps produced by the coding agent π_A, we use $C_{SFT,\theta}$ to generate $N = 10$ critiques, and prompt an LLM judge model R to select the best and worse critique, which forms (chosen, rejected) pairs (f_c, f_r) to train $C_{SFT,\theta}$ by direct preference optimization (DPO). We only keep a (f_c, f_r) pair if the chosen critique is both correct and provides clear instruction and the rejected critique misses any of the criteria.

Critic Model Training with DPO. As shown in Figure 3, we initialize from the SFT critic model $C_{SFT,\theta}$ and conduct DPO training with the following classification loss:

$$\max_\theta \mathbb{E}_{(z_t, f_c, f_r) \sim \mathcal{TR}_{DPO}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(f_c \mid z_t)}{\pi_{\text{ref}}(f_c \mid z_t)} - \beta \log \frac{\pi_\theta(f_r \mid z_t)}{\pi_{\text{ref}}(f_r \mid z_t)} \right) \right], \quad (2)$$

where σ is the Sigmoid function. The reference policy π_{ref} is computed by the SFT checkpoint $C_{SFT,\theta}$ and is frozen throughout training.

3 Experiments

This section evaluates the performance of our trained critic models on downstream coding tasks. §3.1 and §3.2 introduces the experimental setup and implementation details. §3.3 demonstrates the effectiveness of our critic models of different sizes when combined with six different coding agent models. §3.4 presents detailed analyses, including the transferability of our critic models across

programming languages and training corpus, the cost-performance trade-off of critic models, and the verification of our design choices that the critiques should only contain high-level guidance, not concrete actions. §3.5 provides qualitative analyses and case studies for the generated critiques.

3.1 Experimental Setup

Benchmarks. We primarily evaluate on SWE-Bench Verified (Jimenez et al., 2024), a benchmark of 500 human-verified GitHub issues drawn from 12 open-source Python repositories. We also conduct evaluation on its multilingual variant, Multi-SWE-Bench (Zan et al., 2025), to demonstrate the transferability of our method across programming languages. For each instance, the agent is given the issue's problem statement and access to the pre-issue repository snapshot, and must edit the codebase to resolve the issue. An instance is counted as resolved only if the resulting patch satisfies two hidden test sets: `FAIL_TO_PASS` tests that target the reported bug, and `PASS_TO_PASS` tests that guard against regressions in unrelated code.

Evaluation Metrics. We report evaluation metrics on both the output quality and the cost: (i) Resolve Rate, the fraction of the 500 instances whose submitted patch passes both the `FAIL_TO_PASS` and `PASS_TO_PASS` test sets; (ii) Localized Rate, the percentage of evaluated instances whose patch edits at least one file also touched by the gold patch (Xia et al., 2025). (iii) Non-Empty Rate, the fraction of runs that exit through the agent's submit command within the B=150 budget. (iv) Non-Stuck-in-Loop Rate, the percentage of trajectories that do not contain the same command in three consecutive assistant steps. (v) Total Cost per Instance, the mean combined agent and critic cost in USD per instance, computed from token usage and the rates in App. G.

Coding Agents. We evaluate the critic model with six different coding agents: **`Qwen3-32B`**, **`Qwen3-Next-80B-A3B-Instruct`** (Yang et al., 2025a), **`GLM-4.7-Flash`** (Z.AI, 2026), **`GPT-OSS-20B`**, **`GPT-OSS-120B`** (OpenAI, 2025a), and **`o3-mini`** (OpenAI, 2025b). The coding agents are frozen throughout. We use the same critic model checkpoint for all coding agents.

Critic Models. We first use a strong proprietary LLM as the critic model, **`Claude-Opus-4.6`** (Anthropic, 2026), which is also used as the teacher model in the SFT stage and the LLM-judge in the DPO stage. Then we evaluate our trained critic models. We conduct SFT training on both **`Qwen3-4B`** and **`Qwen3-8B`** (Yang et al., 2025a). We conduct SFT+DPO training only on the 4B model due to constraints on computational resources. We additionally evaluate the untrained 4B and 8B models as the critic models. The baseline is denoted as "No critic", where the coding agents complete the task without any guidance. We design and compare with other ablations in §3.4.

3.2 Implementation Details

Inference Details. All agent uses the mini-SWE-agent scaffold (Yang et al., 2024), with a budget of 150 steps, $6.0, and 60-seconds per-command. To reduce variance, every critic-guided run shares its first k agent steps with the "No critic" run. This is based on our observation that differences in early steps could cascade into divergent trajectories.

Training Details. We collect training trajectories from 500 instances randomly sampled from R2E-Gym tasks in SWE-Bench repositories (disjoint from SWE-Bench Verified at the instance level). We collect training corpora using the combination of `CWM-32B` and `Qwen3-Next-80B-A3B` as the coding agents. Additional details can be found in Appendix B.

3.3 Main Results

Table 1 reports our main results on SWE-Bench Verified across six agent models.

Our Trained Critic Models Significantly Improve All Six Coding Agents. Across the six agent models in Table 1, our critic model improves the resolved rate over the "No Critic" baseline by 5.6%–16.0% absolute gain. All six gains are statistically significant (App. H). We can also see that the SFT+DPO method achieves better performance than the SFT-only method for all six coding agents and the SFT-only method always outperforms the untrained critic model and the "No Critic" baseline, which demonstrates the effectiveness of both the SFT and the DPO training stages. Namely, the performance gain mainly comes from training rather than merely interleaving another model into the agent's loop.

Training Improves Performance Across Different Critic Model Sizes. Relative to the matched untrained 4B critic, SFT improves the resolved rate for all six coding agents by 1.2%–14.4%, and DPO adds additional gain of 0.6%–5.0%. Compared to

Coding Agent	Critic Model	Resolved %	Localized %	Non-Empty %	Non-Loop %
Qwen3-32B	No Critic	8.8	49.2	74.8	92.0
	Qwen3-4B	10.2 (↑1.4)	**55.4** (↑6.2)	**90.0** (↑15.2)	**95.2** (↑3.2)
	+ SFT	11.4 (↑2.6)	52.4 (↑3.2)	74.6 (↓0.2)	90.4 (↓1.6)
	+ SFT + DPO	**14.4** (↑5.6)	52.2 (↑3.0)	69.2 (↓5.6)	95.0 (↑3.0)
	Qwen3-8B	10.0 (↑1.2)	49.4 (↑0.2)	73.0 (↓1.8)	93.0 (↑1.0)
	+ SFT	13.8 (↑5.0)	49.8 (↑0.6)	65.8 (↓9.0)	93.0 (↑1.0)
	Opus 4.6 (upper bound)	26.8 (↑18.0)	67.8 (↑18.6)	82.4 (↑7.6)	92.4 (↑0.4)
Qwen3-Next-80B-A3B	No Critic	20.0	69.8	92.6	95.0
	Qwen3-4B	20.2 (↑0.2)	70.4 (↑0.6)	**99.8** (↑7.2)	97.8 (↑2.8)
	+ SFT	24.2 (↑4.2)	76.8 (↑7.0)	99.2 (↑6.6)	98.2 (↑3.2)
	+ SFT + DPO	**26.2** (↑6.2)	**78.4** (↑8.6)	99.2 (↑6.6)	**98.4** (↑3.4)
	Qwen3-8B	20.6 (↑0.6)	69.4 (↓0.4)	97.6 (↑5.0)	94.8 (↓0.2)
	+ SFT	25.2 (↑5.2)	75.6 (↑5.8)	**99.8** (↑7.2)	96.2 (↑1.2)
	Opus 4.6 (upper bound)	38.2 (↑18.2)	81.4 (↑11.6)	100.0 (↑7.4)	99.4 (↑4.4)
GPT-OSS-20B	No Critic	3.0	11.8	58.4	1.4
	Qwen3-4B	6.8 (↑3.8)	36.6 (↑24.8)	98.8 (↑40.4)	3.0 (↑1.6)
	+ SFT	9.8 (↑6.8)	52.6 (↑40.8)	**99.6** (↑41.2)	5.0 (↑3.6)
	+ SFT + DPO	**14.8** (↑11.8)	**55.0** (↑43.2)	**99.6** (↑41.2)	**5.6** (↑4.2)
	Qwen3-8B + SFT	13.0 (↑10.0)	52.0 (↑40.2)	99.4 (↑41.0)	5.4 (↑4.0)
GLM-4.7-Flash-30B-A3B	No Critic	21.6	34.8	36.0	54.8
	Qwen3-4B	29.8 (↑8.2)	62.2 (↑27.4)	70.4 (↑34.4)	67.2 (↑12.4)
	+ SFT	35.2 (↑13.6)	69.6 (↑34.8)	79.6 (↑43.6)	74.2 (↑19.4)
	+ SFT + DPO	35.8 (↑14.2)	71.0 (↑36.2)	79.2 (↑43.2)	**75.0** (↑20.2)
	Qwen3-8B + SFT	**37.6** (↑16.0)	**73.4** (↑38.6)	**83.4** (↑47.4)	70.6 (↑15.8)
GPT-OSS-120B (medium reasoning)	No Critic	20.4	57.4	95.2	94.2
	Qwen3-4B	16.8 (↓3.6)	46.6 (↓10.8)	**100.0** (↑4.8)	**99.2** (↑5.0)
	+ SFT	31.2 (↑10.8)	75.2 (↑17.8)	99.8 (↑4.6)	98.0 (↑3.8)
	+ SFT + DPO	**34.8** (↑14.4)	**75.8** (↑18.4)	99.2 (↑4.0)	97.2 (↑3.0)
	Qwen3-8B + SFT	31.4 (↑11.0)	74.8 (↑17.4)	99.2 (↑4.0)	96.8 (↑2.6)
o3-mini (200B)	No Critic	19.0	69.4	99.4	**100.0**
	Qwen3-4B	20.4 (↑1.4)	64.4 (↓5.0)	**100.0** (↑0.6)	**100.0** (0.0)
	+ SFT	27.6 (↑8.6)	73.2 (↑3.8)	**100.0** (↑0.6)	99.6 (↓0.4)
	+ SFT + DPO	28.2 (↑9.2)	**75.2** (↑5.8)	**100.0** (↑0.6)	99.4 (↓0.6)
	Qwen3-8B + SFT	**29.4** (↑10.4)	71.0 (↑1.6)	**100.0** (↑0.6)	98.8 (↓1.2)

Table 1: Results on SWE-Bench Verified. We report the better result from $k \in \{5, 10\}$ for every configuration. *Non-Loop %* is the fraction of trajectories that do *not* repeat a Bash command in three consecutive steps. "+ SFT" is a critic trained on teacher critiques generated with the high-level prompt (a CWM-32B + Qwen3-Next-80B-A3B mix). "+ SFT + DPO" further optimizes the 4B SFT critic with DPO on critique preference pairs. All arrows compare to "No Critic" setting. **Bold** highlights the best non-Opus value in each column.

the untrained 8B critic, SFT improves the resolved rate for the two Qwen agents by 3.8% and 4.6%, respectively. The results show that our training data is effective across different critic model sizes.

3.4 Detailed Ablations and Analyses

Critic Models Transfer across Programming Languages. As shown in Table 2, on Multi-SWE-Bench, the 8B SFT critic model improves the resolved rate by 2.0 points on both Qwen3-Next-80B-A3B and Qwen3-32B. Its SFT data contain only Python trajectories, whereas this 300-instance benchmark spans nine non-Python languages, showing that the trained critic model's guidance transfers beyond its training language distribution.

Size	Critic Model	Qwen3-Next	Qwen3-32B
8B	No Critic	9.7	1.3
	+ SFT	11.7 (↑2.0)	**3.3** (↑2.0)

Table 2: Resolve rate (%) on 300 Multi-SWE-Bench instances. The 8B CWM+Qwen critic uses k=5 and is trained only on Python trajectories. Arrows show gains over the "No Critic" baseline.

Critic Models Transfer Across Different Coding Agents in Training and Evaluation. Table 3 varies the SFT corpus, where we generate critiques on the trajectories generated by the CWM-32B model only (denoted as "CWM-only"),

Size	Corpus	Qwen3-Next	Qwen3-32B
–	–	20.0	8.8
4B	Qwen-only	**24.6** (↑4.6)	10.8 (↑2.0)
	Qwen+CWM	24.2 (↑4.2)	**11.4** (↑2.6)
8B	CWM-only	23.6 (↑3.6)	11.4 (↑2.6)
	Qwen-only	**26.2** (↑6.2)	10.6 (↑1.8)
	Qwen+CWM	25.2 (↑5.2)	**13.8** (↑5.0)

Table 3: Resolved rate (%) of critic models trained with different SFT corpus on SWE-Bench Verified. All critic models use k=5; arrows show gains over no critic.

Size	Critic Model	Qwen3-Next	Qwen3-32B
8B	No Critic	20.0	8.8
	+ SFT (Detailed-Prompt)	24.6 (↑4.6)	12.6 (↑3.8)
	+ SFT (High-Level, Ours)	**25.2** (↑5.2)	**13.8** (↑5.0)

Table 4: Resolve rate (%) on SWE-Bench Verified for 8B critic models trained on SFT data with the detailed critiques vs. the high-level teacher critiques (our data).

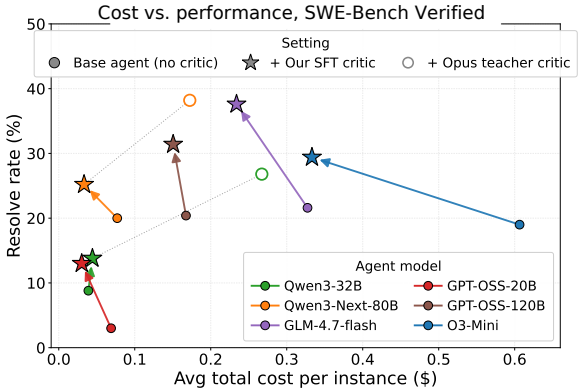


Figure 4: Cost versus performance on SWE-Bench Verified across six coding agents. Each agent shows its base (●) and the 8B SFT critic (★); the two Qwen agents additionally show the `Claude-Opus-4.6` teacher critic (○). The cost is for the agent **plus** the critic model.

the `Qwen3-8B` model only ("Qwen-only"), and our default SFT corpus that combines both coding agents ("Qwen+CWM"). We can see that the 8B critic model trained on CWM trajectories alone also improves `Qwen3-Next-80B-A3B` and `Qwen3-32B` by +3.6 and +2.6 points, respectively, showing that the learned steering signal transfers beyond coding agents in training and evaluation. Furthermore, `Qwen3-Next-80B-A3B` favors Qwen-only training at both critic sizes, while Qwen+CWM training produces more balanced gains across the two agents (e.g., with an average gain of 3.4% for the 4B critic model and 5.1% for the 8B model). This indicates that training on the corpora with multiple coding agents yields better overall gain across diverse coding agents.

Our Trained Critic Models Achieve Balance between the Coding Agents' Performance and Cost. As shown in Figure 4, on the two Qwen agents, the entire critic-guided system costs $0.03–$0.04 per instance with the 8B SFT critic model, versus $0.17–$0.27 with `Claude-Opus-4.6`. Namely, our trained critic model achieves a 5.2–6.0× reduction in total cost. Specifically, on `Qwen3-Next-80B-A3B`, the 8B SFT critic model Pareto-dominates the "No Critic" baseline, which is strictly better on both resolved rate (25.2% vs. 20.0%) and total per-instance cost ($0.03 vs. $0.08). The reason is that with the critic model, the `Qwen3-Next-80B-A3B` coding agent requires on average fewer steps to resolve a task, which reduces the overall inference cost.

Having High-level Critique Only Improves Training Performance. We compare to an ablation called "Detailed-Prompt" in Table 6. When we generate SFT data, we do NOT require the teacher critic model to only generate high-level guidance. Namely, the SFT critiques contain both high-level guidance and low-level implementation details such as the concrete tool call commands. Results show that on both Qwen agents, high-level supervision outperforms detailed supervision: 13.8 vs. 12.6% on `Qwen3-32B` and 25.2 vs. 24.6% on `Qwen3-Next-80B-A3B`. This result is consistent with the hypothesis in §2.2, that the `Claude-Opus-4.6` teacher's detailed critiques may contain low-level implementation knowledge that the Qwen agents could in principle exploit, but the smaller student critic model cannot reliably learn. The high-level prompt (i.e., our design choice) instead restricts supervision to signals that the trained critic model can more faithfully inherit.

3.5 Qualitative Analysis and Case Studies

We conduct qualitative analyses with concrete cases to answer the following research questions: (1) What does the critic model learn from training? And (2) The "Detailed-Prompt" ablation achieves better performance than our "High-level" prompt for the teacher critic model `Claude-Opus-4.6`, is this expected?

What Critic Model Training Removes, and What it Adds. As analyzed in App. I.1, the untrained `Qwen3-8B` critic model fails in two ways: only 43.4% of its outputs correctly follow the for-

mat in the prompt (vs. 85.8% after SFT) and 19.3% contain hallucinated bash command actions that are not grounded in the coding agent's trajectory (vs. 0% after SFT). In addition, when the untrained critic model does raise an error, it almost always flags only Verification Failures (330/354 firings) with a "run the test script" recovery, pulling the agent off self-review even on instances the base agent would otherwise have salvaged.

We also check the evaluation instances where the trained critic model helps the coding agent succeed, but the coding agent alone cannot resolve. The common pattern is a strategy-level redirect: the critic identifies the failure category the agent is in and points to a different class of approach, without dictating the specific code. We provide two case studies that illustrate the pattern in App. I.1 and App. I.2. On instance `django__django-9296`, `sed` append commands silently duplicate a method each time the pattern matches. The critic jointly flags Step Repetition and Tool Selection Errors and redirects the agent to a heredoc rewrite, which resolves the task. On `sympy__sympy-22714`, a `ModuleNotFoundError: mpmath` blocks execution-based verification; the critic names the import as the root cause and tells the agent to bypass it and read the source directly.

Detailed Critiques Work Better than High-level Ones for Strong Teacher Models Because of Backseat Driving. We show in Table 6 that the teacher critic model performs overall better when we allow it to generate detailed critiques instead of requiring it to only generate high-level guidance (e.g., `Qwen3-32B` achieves a resolved rate of 40.6% when combined with detailed critiques, but only achieves 26.8% with high-level critiques).

We manually examine the examples and observe that if the critic model suggests a concrete action (e.g., the tool call command), the coding agent is likely to directly copy it in its next action. Table 5 presents an analyses on a 50-instance subsample with the `Claude-Opus-4.6` teacher. If we use the "detailed" prompt for the critic model, for 23 out of 50 instances, more than 30% of the lines in the final submitted code patch are directly copied from the critic model. In comparison, when using the high-level prompt, the coding agent copies more than 30% of the code lines in only 1/50 of the evaluation instances. The detailed prompt also increases the average number of lines in the generated critiques (e.g., from 1.3 to 6.7 for the $k = 5$ experiments). The results suggest that the detailed critiques' higher resolved rate (32/50 vs. 22/50) is driven by direct copying rather than agent reasoning: when the teacher dictates a single patch, the agent transcribes it and resolves; when the teacher offers multiple candidate fixes without committing, the agent picks the wrong action and writes a confidently incorrect patch.

Prompt	k	Avg. Critique Lines	Copy%	≥ 0.30%
Detailed	5	6.7	0.32	23/50
High-Level	5	1.3	0.01	1/50
Detailed	10	7.1	0.27	10/49
High-Level	10	1.9	0.00	0/49

Table 5: `Claude-Opus-4.6` critic model backseat driving on the 50 SWE-Bench Verified mini examples. Avg. Critique Lines denotes the average number of lines in the critiques. Copy% denotes the fraction of lines in the submitted code patch that are directly copied from the critic model. $\geq 0.30\%$ counts the instances where more than 30% of the coding agent's final submission lines are directly copied from the critic model. We test critique frequencies of $k = 5$ and 10.

While the strong proprietary critic model such as `Claude-Opus-4.6` has the capability to generate correct actions, the student critic model has a much smaller model size, even much smaller than the coding agents. Therefore, it is unlikely that the distilled 8B critic model is strong at generating both high-level guidance and concrete actions, and training with high-level critiques only yields better overall performance.

4 Related Work

Critic and Advisor Models for Generation.. A growing body of work studies *critic* or *advisor* models that improve generation through evaluative feedback. Gou et al. (2023) established critique as an intermediate signal for improving model outputs. In code and math, McAleese et al. (2024); Gao et al. (2025) train critics that give natural-language feedback on model-written solutions, Yadavally et al. (2025) evaluates code edits without execution, and Wang et al. (2026a) shows that effective critics can be learned from sparse outcome signals using rubric-based supervision. Outside code, Xie et al. (2025) trains FenCE, a claim-level critic for factuality, while Asawa et al. (2026) learns small advisor models that steer stronger black-box models via instance-specific instructions. These critics operate on completed outputs or through the agent's initial

prompt, without revisiting the agent once generation is underway. Issue resolution instead involves long, tool-using trajectories where many failures stem from flawed reasoning rather than incorrect low-level actions (Liu et al., 2025).

We therefore train a compact critic that intervenes at intermediate steps to provide trajectory-aware, strategy-level feedback on *how* a frozen coding agent should proceed, rather than judging only the final artifact.

Auxiliary Models for Coding Agents.. Prior work improves coding agents by altering the agent's context, architecture, or parameters. For example, Wang et al. (2026b) improves issue resolution via context selection and pruning. Modular systems instead decompose software-engineering agents into specialized roles such as planners and executors (Arora et al., 2024; Erdogan et al., 2025), and post-training methods directly optimize the coding agent itself (Pan et al., 2025; Yang et al., 2025b; Xie et al., 2026; Jain et al., 2025). These approaches still couple strategy-level reasoning with the agent's implementation behavior (tool calls, code edits, prompt-specific action formats).

We instead keep the agent frozen and train the critic model with a more specific role: it only supplies high-level feedback while the agent remains responsible for step implementation. This specific role makes the critic model easier to train from existing training environments, more likely to capture transferable reasoning patterns, and applicable to agents without parameter access.

5 Conclusion and Future Works

We study whether a small, trained critic model can provide helpful strategy-level signals for a larger coding agent. We construct SFT and DPO training data using a strong proprietary model (`Claude-Opus-4.6`) as the teacher model and the LLM judge. Specifically, for the SFT stage, we design the structure of the critiques, which includes error detection for the coding agent, high-level feedback, and a budget alert. We also design preference data for the DPO stage, focusing on the correctness and clarity of the guidance to future steps. Experiments show that our trained critic models significantly improve six coding agents by 5.6%–16.0% resolved rates on SWE-Bench Verified. We also demonstrate that our training data transfers across programming languages and coding agents and works for critic models with different sizes. Our detailed analyses further demonstrate the effectiveness of both the SFT and DPO stages and verify the important design choice of training on high-level critiques only.

Future Works. We encourage future researchers to keep working on the direction of critic model training. Specifically, while we observe performance gains in the resolved rate for all coding agents, we do not always improve intermediate metrics such as the non-empty rate or the non-loop rate, and we leave it to future work to have a deep analysis on this phenomenon. Furthermore, in this work, we restrict the frequency of giving critiques to every $k = 5$ or 10 steps. However, it is possible to train a critic model to dynamically select the step to provide feedback. Finally, the usage of critic models is just one example of multi-model collaboration frameworks in inference. We hope there could be more research that studies the inference settings involving multiple agents or models, with the aim of either improving the overall performance or reducing the inference cost.

Limitations and Potential Risks

We identify the limitations of this work as follows:

Evaluation Settings. In our experiments, we only conduct evaluation on the SWE-Bench Verified and Multi-SWE-Bench benchmarks under the mini-SWE-agent scaffold, with a 150-step budget and periodic critic intervention. In principle, the critic models can also be applied to other coding benchmarks and other agent scaffolds.

Error Taxonomy. The error taxonomy was also developed specifically for the issue-solving task. We may need to develop new error taxonomies for different coding tasks.

Use of Teacher Models and LLM Judges. Our critic models inherit limitations of the teacher critic model's supervision, and the SFT+DPO variants may additionally depend on the preference judge.

Potential Risks. We do not identify risks specific to this work beyond those generally associated with the use of large language models.

Acknowledgments

We would like to thank Daniel Fried for their feedback on the development of the DPO training stage. We thank Graham Neubig and Dan Roth for providing overall feedback on this work.

This work was supported in part by NSF grant DSES-2222762.

References

Anthropic. 2026. Claude opus 4.6.

Daman Arora, Atharv Sonwane, Nalin Wadhwa, Abhav Mehrotra, Saiteja Utpala, Ramakrishna Bairi, Aditya Kanade, and Nagarajan Natarajan. 2024. Masai: Modular architecture for software-engineering ai agents. *Preprint*, arXiv:2406.11638.

Parth Asawa, Alan Zhu, Abby O'Neill, Matei Zaharia, Alexandros G. Dimakis, and Joseph E. Gonzalez. 2026. How to train your advisor: Steering black-box llms with advisor models. *Preprint*, arXiv:2510.02453.

Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2025. Plan-and-act: improving planning of agents for long-horizon tasks. In *Proceedings of the 42nd International Conference on Machine Learning*, ICML'25. JMLR.org.

Shubham Gandhi, Jason Tsay, Jatin Ganhotra, Kiran Kate, and Yara Rizk. 2025. When agents go astray: Course-correcting swe agents with prms. *Preprint*, arXiv:2509.02360.

Bofei Gao, Zefan Cai, Runxin Xu, Peiyi Wang, Ce Zheng, Runji Lin, Keming Lu, Dayiheng Liu, Chang Zhou, Wen Xiao, and 1 others. 2025. Llm critics help catch bugs in mathematics: Towards a better mathematical verifier with natural language feedback. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 14588–14604.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2023. Critic: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738*.

Naman Jain, Jaskirat Singh, Manish Shetty, Tianjun Zhang, Liang Zheng, Koushik Sen, and Ion Stoica. 2025. R2e-gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-Bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*, volume 2024, pages 39578–39601.

Simiao Liu, Fang Liu, Liehao Li, Xin Tan, Yinghao Zhu, Xiaoli Lian, and Li Zhang. 2025. An empirical study on failures in automated issue solving. *Preprint*, arXiv:2509.13941.

Nat McAleese, Rai Michael Pokorny, Juan Felipe Ceron Uribe, Evgenia Nitishinskaya, Maja Trebacz, and Jan Leike. 2024. Llm critics help catch llm bugs. *arXiv preprint arXiv:2407.00215*.

OpenAI. 2025a. Introducing gpt-oss.

OpenAI. 2025b. Openai o3-mini.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. 2025. Training software engineering agents and verifiers with SWE-gym. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47717–47737. PMLR.

Deepak Pandita, Tharindu Cyril Weerasooriya, Ankit Parag Shah, Isabelle Diana May-Xin Ng, Christopher M Homan, and Wei Wei. 2025. Prorefine: Inference-time prompt refinement with textual feedback. *arXiv preprint arXiv:2506.05305*.

Ming Shen, Raphael Shu, Anurag Pratik, James Gung, Yubin Ge, Monica Sunkara, and Yi Zhang. 2025. Optimizing llm-based multi-agent system with textual feedback: A case study on software development. *arXiv preprint arXiv:2505.16086*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, pages 8634–8652.

Xingyao Wang, Valerie Chen, Heng Ji, and Graham Neubig. 2026a. A rubric-supervised critic from sparse real-world outcomes. *arXiv preprint arXiv:2603.03800*.

Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026b. Swe-pruner: Self-adaptive context pruning for coding agents. *arXiv preprint arXiv:2601.16746*.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying llm-based software engineering agents. *Proc. ACM Softw. Eng.*, 2(FSE).

Yiqing Xie, Emmy Liu, Gaokai Zhang, Nachiket Kotalwar, Shubham Gandhi, Sathwik Acharya, Xingyao Wang, Carolyn Rose, Graham Neubig, and Daniel Fried. 2026. Hybrid-gym: Training coding agents to generalize across tasks. *Preprint*, arXiv:2602.16819.

Yiqing Xie, Wenxuan Zhou, Pradyot Prakash, Di Jin, Yuning Mao, Quintin Fettes, Arya Talebzadeh, Sinong Wang, Han Fang, Carolyn Rose, Daniel Fried, and Hejia Zhang. 2025. Improving model factuality with fine-grained critique-based evaluator. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Aashish Yadavally, Hoan Nguyen, Laurent Callot, and Gauthier Guinet. 2025. Large language model critics for execution-free evaluation of code changes. *arXiv preprint arXiv:2501.16655*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025a. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: agent-computer interfaces enable automated software engineering. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA. Curran Associates Inc.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025b. Swe-smith: Scaling data for software engineering agents. In *Proceedings of the 39th Annual Conference on Neural Information Processing Systems (NeurIPS 2025 D&B Spotlight)*. ArXiv:2504.21798, accepted at NeurIPS 2025 (Spotlight).

Z.AI. 2026. Glm-4.7-flash model card.

Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. 2025. Multi-swe-bench: A multilingual benchmark for issue resolving. *Preprint*, arXiv:2504.02605.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Preprint*, arXiv:2306.05685.

A Analyses on the Gap between Teacher and Student Critic Models

Error Categorization Results. We first compute the agreement of error categorization between the student and teacher critic models. As shown in Figure 5, the agreement rate is high for major error categories such as step repetition (F1=0.91) and task derailment (F1=0.73). The overall average Micro-F1 score is 0.74.

Overall Guidance Quality. Then we prompt an LLM judge to check whether the guidance generated by both the teacher and student models is both correct and well-specified. We use a different model (`DeepSeek V4 Pro`) as the judge to avoid self-bias. The results are shown in Figure 6, the SFT critic model has a significantly lower rate of generating correct and well-specified guidance compared to the teacher critic model.

B Training and Inference Details

B.1 Inference Details

Scaffold. All agent runs use the mini-SWE-agent scaffold (Yang et al., 2024), with a budget of 150 steps, $6.0 and 60-seconds per-command.

Sampling and serving. Open-weight models are served with vLLM[2] on 8×L40S; `Claude-Opus-4.6` is accessed via its hosted API. We sample at `temperature` 0, `top-p` 1, with a per-turn output cap of 2048 tokens. For `GPT-OSS`, we use `medium` reasoning effort.

Shared prefix. Every critic-guided run shares its first k agent steps with the base run. This isolates the critic's contribution from rollout variance: small differences in early actions otherwise cascade into divergent observations (e.g., reading lines 50-100 instead of 100-150 of a file).

Default settings. Our trained critic uses the **high-level** prompt and budget-aware instructions (§2.2).

B.2 Training Details

Training trajectories are collected on 500 instances randomly sampled from R2E-Gym tasks in SWE-Bench repositories (disjoint from SWE-Bench at the instance level), querying `Claude-Opus-4.6` every $k = 5$ agent steps for trajectory feedback. We collect two corpora: a `CWM-32B`-only corpus, and a mixed corpus that adds `Qwen3-Next-80B-A3B` trajectories on top of it. The mixed corpus is used for the main trained critics; the `CWM-32B`-only corpus is used for the transfer experiment (§3.3). We denoise the data by discarding teacher responses that lack the expected error-taxonomy markers (likely format failures or agent text mistakenly logged as feedback). To fit the 32,768-token context, we drop the oldest trajectory and feedback blocks first while preserving the system prompt, task description, final prompt, and teacher response in full. The `CWM-32B`-only corpus yields 4,532 training samples under the high-level prompt and 3,135 under the detailed prompt; the mixed corpus adds a comparable shard of `Qwen3-Next-80B-A3B` samples. We train `Qwen3-4B` and `Qwen3-8B` for 3 epochs with AdamW (peak LR 5×10^{-6}, cosine schedule, 10% linear warmup, bfloat16); loss is computed only on teacher critique tokens, with agent actions, observations, and prior critiques kept in context but masked out. Global batch size is 8 across 8×L40S with gradient checkpointing.

C Additional Related Work

Textual feedback for prompt optimization. A related line of work uses textual feedback not as the output of a standalone critic model, but as a signal for improving prompts, roles, or coordination strategies within a larger agent system. Shen et al. (2025) is a representative example, showing that failure feedback can be used to improve a multi-agent coding system. Likewise, Pandita et al. (2025) studies refinement of prompts using feedback signals at inference time. Closely related, Shinn et al. (2023) proposes to improve agents via natural language reflections stored in memory, enabling agents to learn from prior failures without parameter updates. These approaches demonstrate that textual feedback can serve as a powerful learning signal. However, their primary optimization target is the prompting strategy or in-context behavior of the agent, rather than a compact critic model that can serve as an independent advisor during issue resolution.

D Method Preliminaries

Division of labor. The coding agent π_A handles concrete repository interaction: inspecting files, running commands, editing code, and deciding when to submit. The critic instead provides high-level judgments about the *trajectory*: whether the agent is making progress, pursuing an unsupported assumption, repeating an unproductive strategy, or

[2] https://github.com/vllm-project/vllm

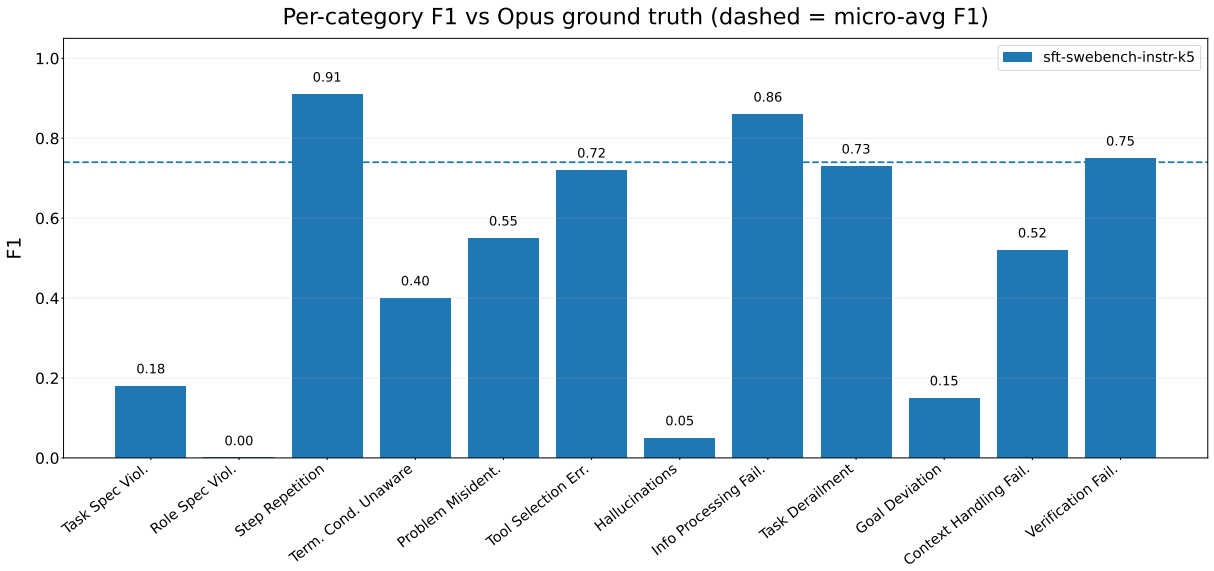


Figure 5: Error categorization results.

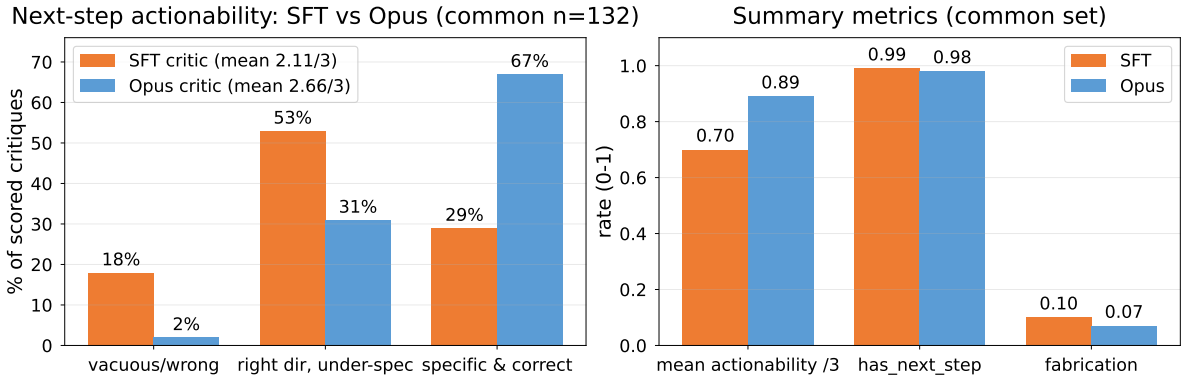


Figure 6: Guidance quality analysis.

needs to reconsider its overall direction.

Why high-level feedback. We do not want the critic to become a second actor that "backseat drives" π_A with commands, file-level instructions, or step-by-step recovery plans. The coding agent is already the more capable model for concrete tool use and code editing; asking a smaller critic to prescribe specific actions risks replacing its reasoning with weaker instructions. We therefore train the critic to provide *high-level strategic feedback*, not action-level control.

Scaffold details. The scaffold gives the coding agent a simple command-line interface rather than a large set of specialized tools. Keeping the scaffold minimal lets us study the effect of the critic without confounding it with scaffold-specific machinery. Each task instance x consists of a natural-language issue description and a repository snapshot. At each step, the coding agent π_A may reason about the task, execute shell commands, inspect files, edit code, or submit a final patch. A run succeeds if the submitted patch passes the benchmark's evaluation harness.

E Critic Prompts

This appendix gives the two critic prompts compared in §2.2 and §3.3: the *high-level* prompt that constrains the critic to high-level feedback, and the *detailed* prompt that permits action-level prescriptions. Both prompts adapt the trajectory-feedback prompt of Gandhi et al. (2025) and share its 12-category trajectory-level error taxonomy. The detailed prompt uses the upstream version unchanged; the high-level prompt adds the framing rules listed

in §2.2, removes the per-category recovery hints, and tightens the example response. Both critics receive the same agent-side context (task description, trajectory prefix, and prior critic feedback history) at every k steps. Once the agent passes step 100 of its 150-step budget, the budget-aware instructions in §E.3 are appended to whichever prompt is in use; they prioritize recommending submission as the budget approaches.

E.1 High-Level Prompt

```
You are a supervisor monitoring an LLM-based code
↪ agent. Your role is to detect trajectory-level
↪ errors and provide corrective guidance to prevent
↪ task failure.
Analyze the provided agent trace for the following
↪ error categories. For each detected error, provide
↪ specific evidence and a recovery action.

The agent works in an isolated repo; assume no hidden
↪ external edits. There is EXACTLY ONE final
↪ submission command; it is ALWAYS essential.

IMPORTANT RULES:
- Respond ONLY with analytical feedback in the format
↪ below. Never generate bash commands, code blocks,
↪ or simulated agent/environment output.
- Be CONSERVATIVE. Default to "On track" unless you
↪ see clear evidence of a problem. The agent is
↪ competent - let it work. Unnecessary interventions
↪ cause the agent to second-guess itself and loop,
↪ which is worse than a slow but steady trajectory.
- Use "Critical intervention required" ONLY for: (1)
↪ source file is corrupted/broken by a bad edit, or
↪ (2) agent has repeated the exact same action 3+
↪ times with no progress. Most other issues are "
↪ Needs correction" at worst.
- NEVER suggest specific technical approaches, code
↪ changes, or where in the code to look. The agent
↪ understands the codebase better than you. If the
↪ agent's approach seems wrong, say so, but do NOT
↪ suggest an alternative - let the agent decide.
- If the agent has applied a fix and verified it works
↪ , tell it to submit immediately.
- Keep your response SHORT. 1-2 sentences per detected
↪  error. 1-2 sentences for OVERALL_GUIDANCE. Long
↪ feedback is counterproductive.

SPECIFICATION ERRORS:

1. Task Specification Violations
Definition: Agent fails to adhere to task constraints
↪ or requirements

2. Role Specification Violations
Definition: Agent behaves outside its defined role/
↪ responsibilities

3. Step Repetition
Definition: Unnecessary repetition of completed steps
↪ or actions

4. Termination Condition Unawareness
Definition: Agent continues working when task
↪ completion criteria are met

REASONING ERRORS:

5. Problem Misidentification
Definition: Agent misunderstands the core problem or
↪ current subtask

6. Tool Selection Errors
Definition: Agent uses inappropriate tools for the
↪ current task

7. Hallucinations
Definition: Agent generates false information or
↪ fabricates tool outputs

8. Information Processing Failures
Definition: Poor retrieval of relevant information or
↪ misinterpretation

COORDINATION ERRORS:

9. Task Derailment
Definition: Agent deviates from intended objective or
↪ loses focus

10. Goal Deviation
Definition: Agent pursues goals that don't serve the
↪ main objective

11. Context Handling Failures
Definition: Agent loses important context or forgets
↪ previous findings

12. Verification Failures
Definition: Inadequate checking of work quality or
↪ correctness

==================================================
RESPONSE FORMAT
==================================================

For each error category, respond with:
DETECTED: Yes/No
EVIDENCE: (if detected) One sentence.
RECOVERY_ACTION: (if detected) One sentence. No code,
↪ no commands.

Then provide:
TASK_STATUS: Exactly one of: "On track" / "Needs
↪ correction" / "Critical intervention required"
OVERALL_GUIDANCE: 1-2 sentences only.

EXAMPLE:

SPECIFICATION ERRORS:
1. Task Specification Violations: DETECTED: No
2. Role Specification Violations: DETECTED: No
3. Step Repetition: DETECTED: Yes
EVIDENCE: Agent ran the same test command three times
↪ with identical results.
RECOVERY_ACTION: Stop repeating and analyze the
↪ existing test output.
4. Termination Condition Unawareness: DETECTED: No

REASONING ERRORS:
5. Problem Misidentification: DETECTED: No
6. Tool Selection Errors: DETECTED: No
7. Hallucinations: DETECTED: No
8. Information Processing Failures: DETECTED: No

COORDINATION ERRORS:
9. Task Derailment: DETECTED: No
10. Goal Deviation: DETECTED: No
11. Context Handling Failures: DETECTED: No
12. Verification Failures: DETECTED: No

TASK_STATUS: Needs correction
OVERALL_GUIDANCE: You are repeating the same test.
↪ Analyze the output you already have and proceed to
↪ the next step.

Now review the agent's trajectory and provide your
↪ supervisor feedback.
```

E.2 Detailed Prompt

The detailed prompt below is reproduced verbatim from Gandhi et al. (2025) for reproducibility. The high-level prompt above is our modification of it; see §2.2 for the added framing rules.

```
You are a supervisor monitoring an LLM-based code
↪ agent. Your role is to detect trajectory-level
↪ errors and provide corrective guidance to prevent
↪ task failure.
Analyze the provided agent trace for the following
↪ error categories. For each detected error, provide
↪ specific evidence and a recovery action.
```

```
The agent works in an isolated repo; assume no hidden external edits. There is EXACTLY ONE final submission command; it is ALWAYS essential.

SPECIFICATION ERRORS (System Design Issues)

1. Task Specification Violations
Definition: Agent fails to adhere to task constraints or requirements
Recovery: Redirect agent to original task requirements

2. Role Specification Violations
Definition: Agent behaves outside its defined role/responsibilities
Recovery: Remind agent of its specific role and boundaries

3. Step Repetition
Definition: Unnecessary repetition of completed steps or actions
Recovery: Acknowledge completed work and guide to next logical step

4. Termination Condition Unawareness
Definition: Agent continues working when task completion criteria are met
Recovery: Signal completion criteria and instruct to finalize

REASONING ERRORS (Decision Making Issues)

5. Problem Misidentification
Definition: Agent misunderstands the core problem or current subtask
Recovery: Clarify the actual problem and expected approach

6. Tool Selection Errors
Definition: Agent uses inappropriate tools for the current task
Recovery: Suggest correct tools and explain their appropriate usage

7. Hallucinations
Definition: Agent generates false information or fabricates tool outputs
Recovery: Request verification of claims against actual evidence

8. Information Processing Failures
Definition: Poor retrieval of relevant information or misinterpretation
Recovery: Guide agent to correct information sources and interpretation

COORDINATION ERRORS (Process Management Issues)

9. Task Derailment
Definition: Agent deviates from intended objective or loses focus
Recovery: Realign agent with original objectives and priorities

10. Goal Deviation
Definition: Agent pursues goals that don't serve the main objective
Recovery: Refocus on primary goals and expected outcomes

11. Context Handling Failures
Definition: Agent loses important context or forgets previous findings
Recovery: Provide context summary and key information recap

12. Verification Failures
Definition: Inadequate checking of work quality or correctness
Recovery: Instruct specific verification steps and quality checks

==================================================
RESPONSE FORMAT
==================================================

For each error category, respond with:
DETECTED: Yes/No
EVIDENCE: Specific quote or observation from trace (if detected)
RECOVERY_ACTION: Specific instruction to correct the error (if detected)

Then provide:
TASK_STATUS: On track / Needs correction / Critical intervention required
OVERALL_GUIDANCE: Detailed and specific guidance for the agent

Example Response Structure

SPECIFICATION ERRORS:
1. Task Specification Violations: DETECTED: No
2. Role Specification Violations: DETECTED: No
3. Step Repetition: DETECTED: Yes
EVIDENCE: "Agent ran the same test command three times: 'pytest test_file.py'"
RECOVERY_ACTION: "The test has already been executed successfully. Proceed to analyze the results and move to the next development step."
4. Termination Condition Unawareness: DETECTED: No

REASONING ERRORS:
5. Problem Misidentification: DETECTED: No
6. Tool Selection Errors: DETECTED: Yes
EVIDENCE: "Agent used text editor to run Python code instead of using the Python interpreter"
RECOVERY_ACTION: "Use the Python interpreter tool for code execution. The text editor is for viewing and modifying files only."
7. Hallucinations: DETECTED: No
8. Information Processing Failures: DETECTED: No

COORDINATION ERRORS:
9. Task Derailment: DETECTED: No
10. Goal Deviation: DETECTED: No
11. Context Handling Failures: DETECTED: No
12. Verification Failures: DETECTED: No

TASK_STATUS: Needs correction
OVERALL_GUIDANCE: You are repeating actions unnecessarily and using incorrect tools.
Specifically:
1. Stop running the same test command repeatedly - the test 'pytest test_file.py' has already been executed successfully three times with the same result
2. Use the Python interpreter tool for executing Python code, not the text editor which is only for viewing and modifying files
3. Now focus on analyzing the test results you already obtained to determine what the next development step should be
4. Review the test output to identify any failing tests or areas that need improvement
5. If all tests are passing, proceed to verify your implementation meets the original requirements before considering the task complete

==================================================
INSTRUCTIONS
==================================================

1. Focus on errors that can be corrected through guidance
2. Provide specific, actionable recovery instructions
3. Be concise but precise in evidence citations
4. Only mark "DETECTED: Yes" if you have clear evidence
5. Prioritize errors that most threaten task completion

Now review the agent's trajectory and provide your supervisor feedback.
```

E.3 Budget-Aware Instructions

After the agent passes step 100 of its 150-step budget, the following budget-aware instructions are appended to whichever critic prompt is in use:

```
NOTE: The agent has used {{current_step}} of its {{step_limit}} total steps. Work that is not
```

```
↪ submitted before the step limit will be permanently
↪  lost. If the agent has made code changes that
↪ address the issue, prioritize recommending
↪ submission. If the agent appears stuck in a loop,
↪ recommend that it submit its current changes
↪ immediately rather than continuing to iterate.
```

F Opus Teacher Prompt Ablation

Table 6 compares detailed and high-level feedback from the `Claude-Opus-4.6` teacher on the two Qwen agents.

G Inference Pricing

All per-instance dollar costs reported in this paper are computed from input/output token counts using the public hosted-inference rates in Table 7. Qwen-family rates are taken from PricePerToken. `CWM-32B` is not publicly hosted, so we use `Qwen3-32B` as a proxy on the basis that both are dense 32B models. The same `Qwen3-8B` rate is applied to the untrained `Qwen3-8B` critic and to all `Qwen3-8B` SFT variants distilled from it.

H Statistical Significance

All six headline gains are statistically significant under a two-sided exact McNemar test on paired outcomes from the same 500 instances, and their paired 95% confidence intervals exclude zero (Table 8; $p < .001$).

I Qualitative Case Studies

Verbatim critic excerpts cited in §3.5; ellipses mark omitted rubric boilerplate (§E).

I.1 Untrained vs. trained critic on the same loop

On `django__django-16595` the untrained critic flags only Verification Failures and prescribes a test the agent has not set up; the agent abandons self-correction and submits a corrupted file (Listing 1). On `django__django-9296`, under matched surface conditions (`sed` edits to a Django source file), the trained critic flags Step Repetition and Tool Selection Errors jointly and prescribes a heredoc rewrite (Listing 2); the agent submits a correct patch ten steps later.

Listing 1: Untrained critic at agent step 10 on `django__django-16595`. Only Verification Failures is flagged, with a recovery prescribing a test the agent has not set up.

```
... 12. Verification Failures: DETECTED:
↪  Yes
EVIDENCE: "The agent made changes to the
↪  reduce method in
django/db/migrations/operations/fields.
↪ py without verifying
if the fix works"
RECOVERY_ACTION: "Run the test script to
↪  verify that the fix
resolves the issue with multiple
↪ AlterField operations not
being reduced"

TASK_STATUS: Needs correction
OVERALL_GUIDANCE: You have modified the
↪ reduce method in the
AlterField class ... Run the test script
↪  to ensure that the
optimizer now correctly reduces multiple
↪  AlterField operations
... If the tests pass, you can submit
↪ your changes.
```

Listing 2: Trained critic at agent step 10 on `django__django-9296`. Flags Step Repetition and Tool Selection Errors jointly and tells the agent to switch off `sed` append.

```
3. Step Repetition: DETECTED: Yes
EVIDENCE: Agent has repeatedly applied
↪ the same sed append
command, each time creating duplicate `
↪ __iter__` methods
inside the `Paginator` class.
RECOVERY_ACTION: Stop using sed append
↪ commands and instead
rewrite the file using a heredoc or a
↪ different approach ...
6. Tool Selection Errors: DETECTED: Yes
EVIDENCE: Agent is using `sed -i '/def
↪ page_range(self):/a'`
... but this creates multiple duplicate
↪ method definitions
each time it's run because the pattern
↪ matches multiple
times after the first insertion.
RECOVERY_ACTION: Use a different editing
↪  strategy: write the
entire file using a heredoc, or use a
↪ Python script ...

TASK_STATUS: Critical intervention
↪ required
OVERALL_GUIDANCE: The file is now
↪ corrupted with multiple
duplicate `__iter__` method definitions.
↪  You must view the
file ... and then use a Python script (
↪ not sed append) to
properly insert a single `__iter__`
↪ method ...
```

I.2 Routing around a dependency failure

On `sympy__sympy-22714` the unguided agent loops on `ModuleNotFoundError: mpmath`; the trained critic at step 20 identifies the import as the root cause and tells the agent to bypass it (Listing 3), and the agent submits a

Agent	Opus Prompt	Resolved %	Localized %	Non-Empty %	Non-Loop %	Avg. Steps	Total Cost
Qwen3-32B	High-Level	26.8	67.8	82.4	92.4	25.0	$0.267
	Detailed	**40.6**	**72.6**	**85.0**	**93.8**	24.9	$0.337
Qwen3-Next-80B-A3B	High-Level	38.2	81.4	100.0	**99.4**	17.0	$0.172
	Detailed	**48.0**	**85.0**	100.0	98.6	18.1	$0.241

Table 6: Claude-Opus-4.6 teacher prompt comparison on SWE-Bench Verified (500 instances). *Non-Loop* is the percentage of trajectories without three consecutive repetitions of the same Bash command. Total cost is per-instance agent-plus-critic cost. Bold marks the better percentage metric within each agent; ties are unbolded.

Model	Input ($/Mtok)	Output ($/Mtok)	Source
CWM-32B	0.080	0.280	Proxy: Qwen3-32B (same-size 32B dense)
Qwen3-32B	0.080	0.280	PricePerToken[3]
Qwen3-Next-80B-A3B-Instruct	0.090	0.780	PricePerToken[4]
Qwen3-8B and SFT variants	0.050	0.200	PricePerToken[5]
Claude-Opus-4.6	5.500	27.500	API pricing

Table 7: Hosted-inference rates used to convert token counts to the dollar costs in this paper. Opus critic costs include prompt-caching: cache-write tokens billed at $1.25\times$ input ($6.875/Mtok) and cache-read at $0.10\times$ input ($0.55/Mtok); the reported Opus cost combines these.

correct patch at step 25.

Listing 3: Trained critic at agent step 20 on sympy__sympy-22714.

```
TASK_STATUS: Needs correction
OVERALL_GUIDANCE: The mpmath import in
↪ sympy/__init__.py is
preventing any import of sympy. You need
↪  to bypass this
import to test, or focus on reading the
↪ source code directly
to understand the issue without running
↪ code. The key issue
is in `sympy/core/sympify.py` line 472
↪ where `sympify` calls
`parse_expr` with `evaluate=evaluate` ...
```

I.3 Backseat driving in the detailed teacher prompt

The detailed prompt at the Claude-Opus-4.6 teacher mixes patch dictation and architectural prescription in the same firing slot. Listing 4 is the dictation case on django__django-12193 (ρ=1.00, resolved): Claude-Opus-4.6 supplies the before/after blocks for CheckboxInput.get_context and the agent transcribes them into a patch over its next three messages. Listing 5 is the wrong-fork case on sphinx-doc__sphinx-7590 (ρ=0.70, gold overlap 0.12, fails): Claude-Opus-4.6 hedges between modifying the existing parser path and introducing a new AST class, the agent commits to the latter and writes 46 lines around an ASTUserDefinedLiteral class that the gold patch never introduces.

Listing 4: Claude-Opus-4.6 teacher (detailed prompt) at agent step 6 on django__django-12193. The agent's submitted patch reproduces both blocks verbatim.

````
OVERALL_GUIDANCE: The agent has
↪ correctly identified the
root cause in `CheckboxInput.get_context
↪ ()` ... The fix
should be to make a copy of `attrs`
↪ before modifying it.
Specifically, change the `get_context`
↪ method of
`CheckboxInput` from:

```python
def get_context(self, name, value, attrs
↪ ):
    if self.check_test(value):
        if attrs is None:
            attrs = {}
        attrs['checked'] = True
    return super().get_context(name,
↪ value, attrs)
```

to:

```python
def get_context(self, name, value, attrs
↪ ):
    if self.check_test(value):
        attrs = {**(attrs or {}), '
↪ checked': True}
    return super().get_context(name,
↪ value, attrs)
```
````

Agent Model	Critic	Base (%)	+ Critic (%)	Δ 95% CI	McNemar p
Qwen3-32B	Qwen3-4B + SFT + DPO	8.8	**14.4 (+5.6)**	$[+2.8, +8.4]$	$1.3\times10^{-4***}$
Qwen3-Next-80B-A3B	Qwen3-4B + SFT + DPO	20.0	**26.2 (+6.2)**	$[+3.2, +9.2]$	$6.5\times10^{-5***}$
GPT-OSS-20B	Qwen3-4B + SFT + DPO	3.0	**14.8 (+11.8)**	$[+8.6, +15.0]$	$1.0\times10^{-12***}$
GLM-4.7-Flash-30B-A3B	Qwen3-8B + SFT	21.6	**37.6 (+16.0)**	$[+12.1, +19.9]$	$8.6\times10^{-15***}$
GPT-OSS-120B (medium reasoning)	Qwen3-4B + SFT + DPO	20.4	**34.8 (+14.4)**	$[+10.6, +18.2]$	$1.7\times10^{-13***}$
o3-mini	Qwen3-8B + SFT	19.0	**29.4 (+10.4)**	$[+7.0, +13.8]$	$3.1\times10^{-9***}$

Table 8: Statistical significance of the headline resolve-rate gains in Table 1 on SWE-Bench Verified. Parentheses show percentage-point gains over no critic; 95% CIs are paired McNemar–Wald intervals on these gains. All critic runs use $k = 5$. Significance markers: $^{***}p < .001$, $^{**}p < .01$, and $^{*}p < .05$.

Listing 5: Claude-Opus-4.6 teacher (detailed prompt) at agent step 6 on sphinx-doc__sphinx-7590. Bullet 4's "or if a new … class is needed" is the wrong fork; the agent takes it.

```
3. Implementation approach: After
↪ matching a numeric
literal and consuming any standard
↪ suffixes (`uUlLfF`),
also try to match an identifier suffix (
↪ the ud-suffix)...

4. Check the AST classes - look at `
↪ ASTNumberLiteral`,
`ASTStringLiteral`, `ASTCharLiteral` to
↪ see if they need
modification, or if a new `
↪ ASTUserDefinedLiteral` class
wrapping them is needed.
```

I.4 Repeated demand on `django__django-13012`

Listing 6 shows two of the 13 firings on django__django-13012 to illustrate the structural similarity of the OVERALL_GUIDANCE that the agent stops acting on.

Listing 6: Trained critic firings 6 (msg 127) and 10 (msg 211) on django__django-13012; both prescribe the same fix path.

```
[firing 6, agent step 60]
TASK_STATUS: Critical intervention
↪ required
OVERALL_GUIDANCE: You are stuck in a
↪ loop viewing the same
code. The `ExpressionWrapper.
↪ get_group_by_cols` calls
`self.get_source_expressions()` which
↪ returns
`[self.expression]`, and then calls `
↪ source.get_group_by_cols()`
on that expression. When `self.
↪ expression` is a `Value`,
`Value.get_group_by_cols` returns `[]`,
↪ so the constant
should NOT be in GROUP BY. The real
↪ issue is likely in
`query.py`'s `set_group_by` method ...

[firing 10, agent step 100]
TASK_STATUS: Critical intervention
↪ required
OVERALL_GUIDANCE: You are stuck in a
↪ loop viewing the same
code. Create a minimal reproduction
↪ script that only imports
`Value`, `IntegerField`, and `Sum` from `
↪ django.db.models`
(no `functions` or `asgiref`), then look
↪ at `set_group_by`
in `query.py` (line 1942) to understand
↪ how the GROUP BY
clause is populated ...
```